CircuitGuard AI Project Report

Predictive maintenance classification for high-voltage circuit breakers

This report explains what CircuitGuard AI does, how its data and models were prepared, and what the test results show. Random Forest has the highest test accuracy and F1 score. Fine-tuned XGBoost has the highest ROC-AUC and recall. The dataset is partly synthetic and intended for research, so these results do not prove how the system would perform on live breakers.

The report follows the current code and saved results. It marks missing evidence clearly, including development time and prediction speed, which the project did not record.

Executive summary

Topic	Finding
Dataset	5,000 rows; 28 input features; binary target
Primary candidate	Fine-tuned XGBoost; 5-fold stratified GridSearchCV
Comparators	Untuned XGBoost, Random Forest, Gradient Boosting, Logistic Regression
Best accuracy / F1	Random Forest: 92.00% accuracy; 89.32% positive-class F1
Best ROC-AUC / recall	Fine-tuned XGBoost: 0.9826 ROC-AUC; 85.25% positive-class recall
Timing evidence	Tuning 21.8 s and total training pipeline 24.25 s recorded; implementation and evaluation durations unavailable

1. Introduction and project objective

Circuit breakers protect electrical systems by opening when a fault occurs. CircuitGuard AI is a software demonstration: it uses recorded breaker readings to predict whether a dataset record is labeled as needing maintenance. The project combines model training and comparison with an interactive dashboard.

The classifier predicts the dataset label Maintenance_required. It does not forecast a time-to-failure, diagnose a specific physical defect, or establish a maintenance interval. The dataset authors describe the records as semi-synthetically derived for research and educational use, so high test metrics do not establish operational reliability on live equipment [1].

Use the output as a demonstration aid. It cannot replace inspection, manufacturer guidance, or a utility's maintenance rules. The project includes the same limitation in its disclaimer.

2. System model and diagrams

There are two parts. During training, the project prepares data, fits models, and checks their scores. During use, the API loads the saved pipeline and models, accepts breaker readings, and returns a prediction and estimated probability. The React webpage sends the readings to the API and displays the answer.

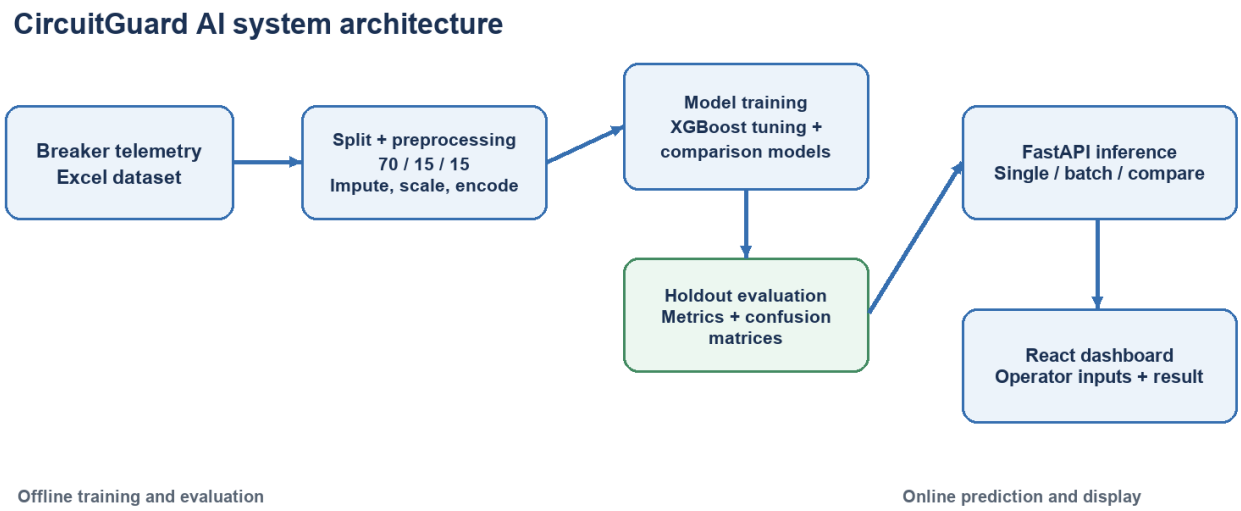


Figure 1. Architecture reconstructed from the project training, backend and frontend structure.

The training process uses a three-way split. The preprocessor is fitted on training data only. The tuned XGBoost search uses training and validation arrays together in five-fold stratified cross-validation, while the holdout test arrays are used by the evaluation script for final scoring.

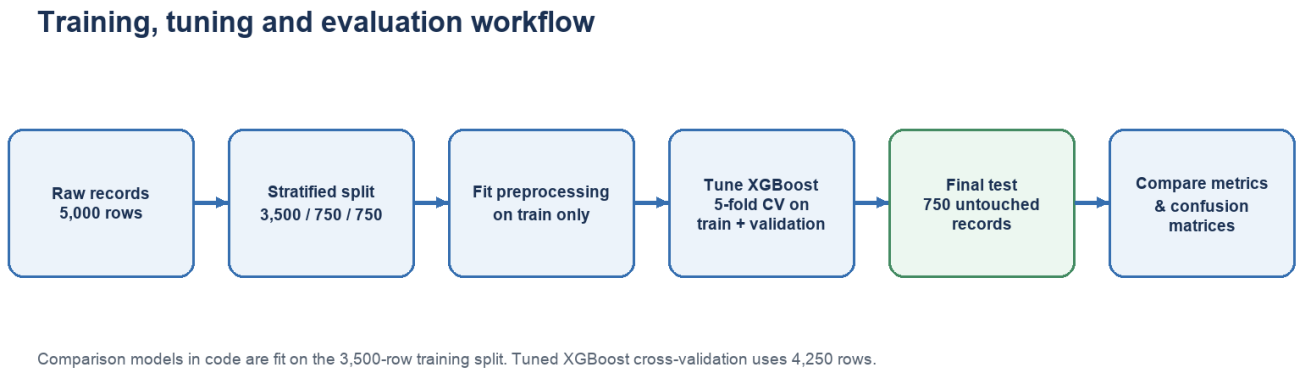


Figure 2. Training, tuning and evaluation workflow. The differing training sample counts are noted in Section 6.

3. Dataset description and provenance

Data source: the workbook HV Circuit Breaker Maintenance Data.xlsx from the repository linked to Ghosh and Mittal's paper [1]. It has 5,000 records grouped into five product variants. The source says the records are partly synthetic and were made for research and education. It also says not every variant has every sensor feature and that maintenance examples were added. Results should therefore be treated as a study of this dataset, not proof about real utility fleets.

The project's data check found 5,000 rows, 29 columns, and no duplicate rows. There are 28 input fields and one target field. The four text/category fields become 11 yes/no columns during preprocessing. Along with the 24 numeric fields, the model receives 35 columns. For example, Product_variant_A means "this row belongs to variant A"; it is not a new sensor reading.

The target is Maintenance_required: 0 means "No Maintenance Required" and 1 means "Maintenance Required." Across all records, 2,965 are class 0 (59.3%) and 2,035 are class 1 (40.7%). The final test set has 445 class 0 records and 305 class 1 records, 750 total.

3.1 Input features

These are the 28 fields given to the model before encoding. Units come from the project's code when available. Category fields show labels rather than physical units. The descriptions are based on field names and the dataset notes; the project did not independently validate each sensor definition.

#	Input feature	Type	Unit / encoding
1	Product_variant	Categorical	Fleet / product group
2	Breaker_status	Categorical	Open / Closed state
3	SF6_density	Numeric	kg/m³
4	SF6_temperature	Numeric	°C
5	SF6_dew_point	Numeric	°C
6	Motor_run_time	Numeric	s
7	Motor_runs_per_day	Numeric	runs/day
8	Motor_I2T	Numeric	A²·s
9	Heater_status	Categorical	On / Off state
10	Line_voltage	Numeric	V
11	Ambient_cabinet_temperature	Numeric	°C
12	Days_since_last_operation	Numeric	days
13	Contact_life	Numeric	points / rating
14	Last_trip_coil_energized	Categorical	TC1 / TC2
15	Clearing_time	Numeric	s
16	Open_latch_time	Numeric	s
17	Close_latch_time	Numeric	s
18	Opening_travel_time	Numeric	s
19	Closing_travel_time	Numeric	s
20	Total_operation_count	Numeric	cycles
21	Non_fault_operation_count	Numeric	cycles
22	Fault_operations_count	Numeric	cycles
23	Accumulated_motor_run_time	Numeric	s
24	Heater_current	Numeric	A
25	Open_average_velocity	Numeric	m/s
26	Close_average_velocity	Numeric	m/s
27	Restrike_occurrence	Numeric flag	0 / 1
28	Open_operation_arcing_time	Numeric	s

3.2 Feature engineering and feature choices

What the code does: it does not calculate new physical measurements such as a temperature difference, a motor-to-operation ratio, or a wear score. It keeps all 28 input fields from the workbook. The only new model columns are yes/no flags made from the four category fields. This is called one-hot encoding.

Original category field	Values turned into flags	New columns
Product_variant	A, B, C, D, E	5
Breaker_status	Open, Closed	2
Heater_status	On, Off	2
Last_trip_coil_energized	TC1, TC2	2
Total category flags		11

Why these inputs were kept: they are the 28 condition fields provided with the dataset, and the project tests whether they help predict the maintenance label. The code does not run a formal feature-selection procedure or remove predictors based on a score. This means the report cannot claim that each input is necessary or optimal. Maintenance_required is left out of the inputs because it is the answer the model is meant to predict; including it would give the model the answer in advance.

Why no other inputs were discarded: the code does not exclude any of the 28 source predictors. Missing values are filled rather than removing columns. The feature-importance chart shows only the 15 largest XGBoost gain scores to make the chart readable; the other inputs still remain in the model. These scores show which inputs the trained trees used for splits. They do not prove that a feature causes maintenance.

The current chart gives large scores to several Product_variant flags, especially C and E, and to Motor_run_time. Product_variant_A is a group label encoded as a yes/no flag, not a measured equipment condition. Because group identity can stand in for differences between fleets, a useful next check is to retrain and test without Product_variant and compare that result with a test that leaves one whole variant out.

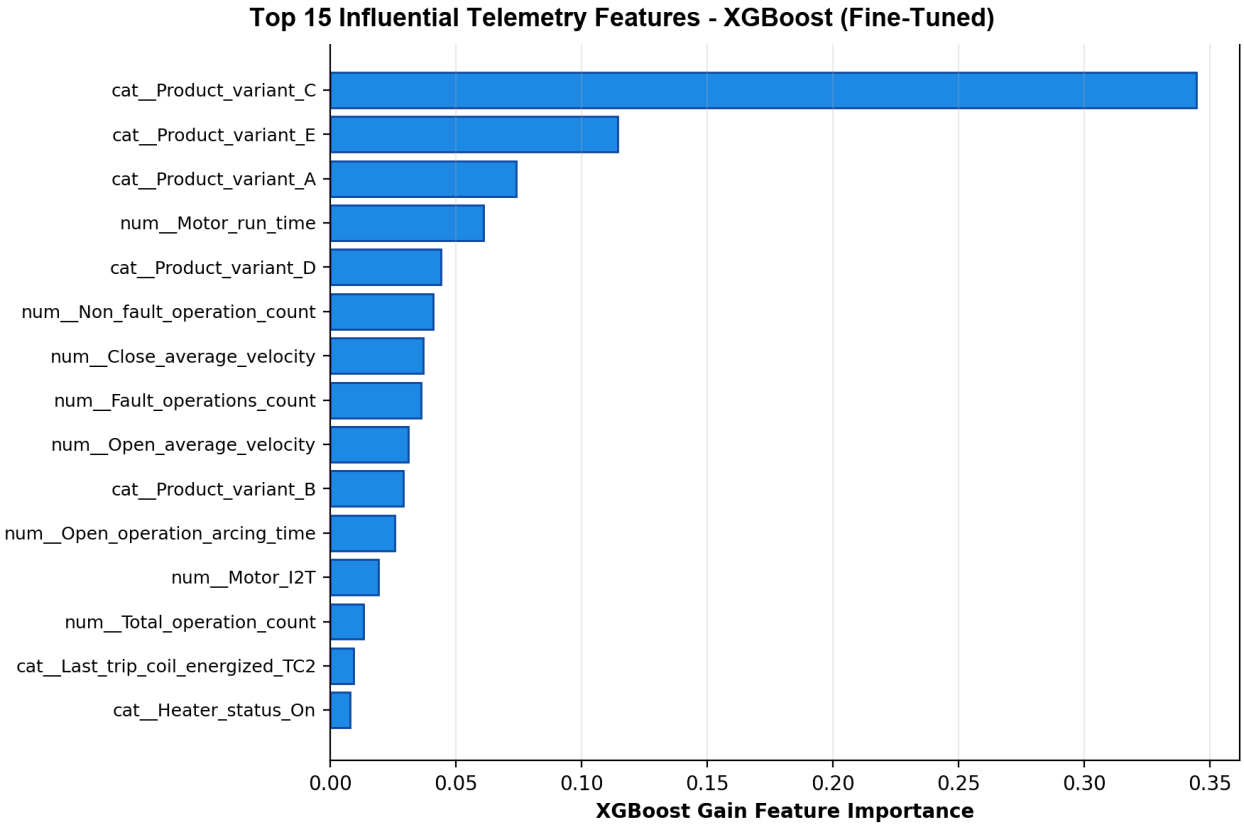


Figure 3. XGBoost gain scores for the 15 inputs shown by the evaluation script. The chart is a ranking aid, not proof of cause or a feature-removal list.

4. Missing variables and treatment

The project found 15,000 empty cells, or 10.34% of the spreadsheet. Eight input fields have missing values. The pattern changes by Product_variant, and the dataset source says some sensor features are not available for every variant [1]. This looks like differences in sensor coverage; the project does not show that the gaps are random.

Variable	Missing rows	Share of 5,000 rows
Days_since_last_operation	1000	20.0%
Clearing_time	1000	20.0%
Accumulated_motor_run_time	2000	40.0%
Heater_current	2000	40.0%
Open_average_velocity	2000	40.0%
Close_average_velocity	2000	40.0%
Restrike_occurrence	2000	40.0%
Open_operation_arcing_time	3000	60.0%

The project fills empty numeric fields with the training-set median (the middle value), then scales numeric values to a common range. It fills empty category fields with the most common training value and turns categories into yes/no columns. These steps are bundled in one saved pipeline. The pipeline is fitted on training data only, then reused for validation, testing, and new predictions. This keeps test data from influencing the fill values or scaling [4].

The project fills missing values; it does not remove rows or fields because they are incomplete. Since missingness differs by product variant, filling values may hide real differences in which sensors are available. A future version should record whether a value is missing because a sensor is not fitted or because a reading was not captured.

5. Methodology

5.1 Split and preprocessing

What we did: the project split the records into 3,500 for training, 750 for validation, and 750 for final testing. It used seed 42 and kept the mix of product variants and target labels similar across the three groups. The saved data-preparation steps were learned from the training group only. This avoids letting the final test records influence the preparation settings.

5.2 XGBoost fine-tuning

What we did: the code tries 216 XGBoost parameter combinations. For each one, it makes five stratified folds, which keep the class mix similar, and scores the model using F1 for class 1 (maintenance required). That means 1,080 model fits in total. The learning rate controls how much each new tree changes the model; max_depth limits tree size; colsample_bytree chooses a subset of input columns for each tree [2], [3].

Parameter	Values searched	Selected
n_estimators	100, 150, 200	200
max_depth	4, 5, 6	6
learning_rate	0.03, 0.05, 0.1	0.05

Parameter	Values searched	Selected
subsample	0.8, 1.0	1.0
colsample_bytree	0.8, 1.0	0.8
min_child_weight	1, 3	1

The selected parameters are 200 estimators, depth 6, learning rate 0.05, subsample 1.0, colsample_bytree 0.8, and min_child_weight 1. The best cross-validation F1 recorded in metadata is 0.8884. Fine-tuned XGBoost is saved as xgboost_tuned.joblib and xgboost.joblib. The XGBoost baseline is configured with 100 estimators, depth 6, learning rate 0.3, subsample 1.0, and full column sampling.

For tuning, the code combines the training and validation groups, giving XGBoost 4,250 rows to use in cross-validation. The comparison models are fit on the 3,500 training rows only. All models are scored on the same 750 test rows and use the same preprocessing, but they did not get the same amount of training data. This makes the comparison less even. A follow-up should give every model the same training records before comparing them.

5.3 Retained models and mathematical basis

Model	Role / mathematical form
Fine-tuned XGBoost	Sequential gradient-boosted trees; tuned model selection by cross-validated F1.
XGBoost baseline	Same family with fixed baseline parameters; isolates the effect of the tuning configuration, subject to training-size caveat.
Random Forest	Bootstrap aggregation of decision trees; a standard nonlinear ensemble comparator.
Gradient Boosting	Sequential additive tree model; standard boosting comparator.
Logistic Regression	Linear log-odds classifier with L2 regularization; interpretable linear baseline.

At a high level, binary classification metrics use the confusion counts: TP = maintenance cases correctly predicted, TN = no-maintenance cases correctly predicted, FP = false maintenance alerts, and FN = missed maintenance cases. Accuracy = (TP+TN)/N; precision = TP/(TP+FP); recall = TP/(TP+FN); and F1 = 2·precision·recall/(precision+recall). ROC-AUC summarizes ranking performance across thresholds; it is not a fixed-threshold accuracy. The project computes binary metrics for class 1 and ROC-AUC/average precision from predicted probabilities [4].

6. Comparative analysis

What the comparison shows: the table uses the project's saved results for the same 750 test records. Precision, recall, and F1 measure performance on class 1, "Maintenance Required." Accuracy is the share of all predictions that were correct. Precision answers "when the model flags maintenance, how often is it right?" Recall answers "of the cases needing maintenance, how many did it find?" F1 balances precision and recall. ROC-AUC measures how well scores rank the two classes across possible cutoffs; it is not the same as accuracy. PR-AUC summarizes precision and recall across cutoffs and focuses on the maintenance class. Both AUC scores use estimated probabilities from the models [4]. Random Forest leads on accuracy and F1. Fine-tuned XGBoost leads on ROC-AUC and recall.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
XGBoost (Fine-Tuned)	91.60%	93.53%	85.25%	89.19%	0.9826	0.9770
XGBoost (Baseline)	91.33%	91.67%	86.56%	89.04%	0.9807	0.9749

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Random Forest	92.00%	97.67%	82.30%	89.32%	0.9794	0.9732
Gradient Boosting (GBM)	91.60%	97.27%	81.64%	88.77%	0.9794	0.9733
Logistic Regression	80.40%	83.47%	64.59%	72.83%	0.9106	0.8900

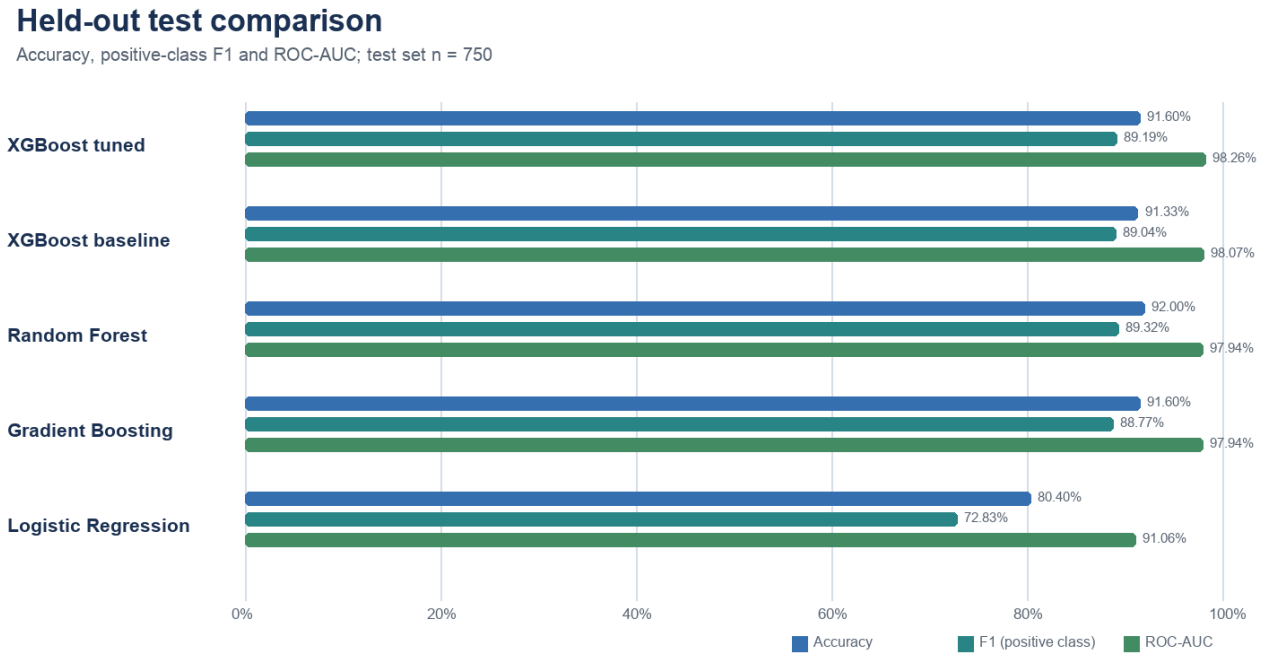


Figure 4. Accuracy, positive-class F1 and ROC-AUC on the common test split.

6.1 Confusion matrices

Each matrix below uses rows for actual classes and columns for predicted classes. Class 0 has support 445 and class 1 has support 305. The matrix counts make the error trade-off visible: Random Forest has only 6 false positives but 54 false negatives; fine-tuned XGBoost has 18 false positives and 45 false negatives.

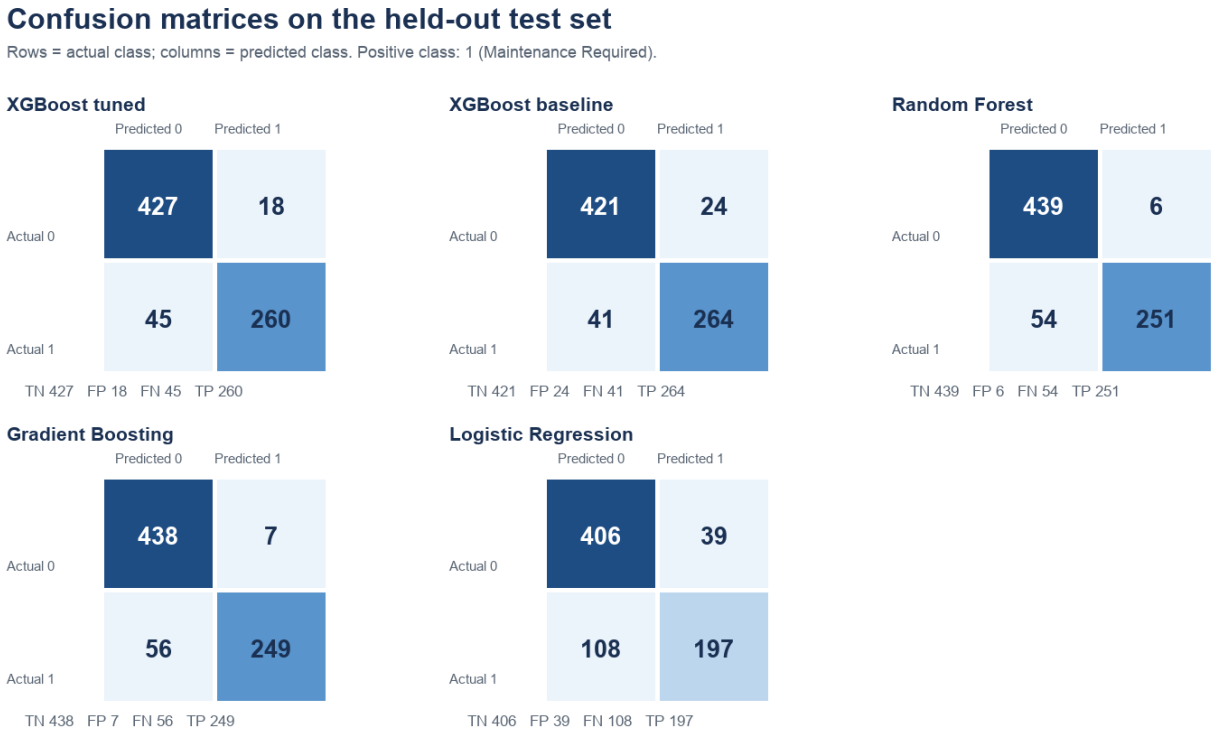


Figure 5. Confusion matrices for all five evaluated models. TN and TP are correct predictions; FP and FN are errors.

6.2 Interpretation and model significance

Fine-tuned XGBoost is a defensible candidate when ranking quality and recall are priorities: its recorded ROC-AUC is 0.9826 and it detects 260 of 305 positive test cases (85.25%). Its F1 is 89.19%, narrowly below Random Forest's 89.32%, and it produces 18 false positives compared with Random Forest's 6. Random Forest is preferable if minimizing false alarms and maximizing the observed accuracy/F1 are the priorities in this test. Thresholds and operational costs should be set with domain experts; the current dashboard does not establish an operational decision threshold.

Compared with the untuned XGBoost baseline, tuned XGBoost changes test accuracy from 91.33% to 91.60%, F1 from 89.04% to 89.19%, ROC-AUC from 0.9807 to 0.9826, precision from 91.67% to 93.53%, and recall from 86.56% to 85.25%. FP decreases from 24 to 18 while FN increases from 41 to 45. This is a small mixed trade-off, not a universal improvement across every metric. The unequal number of rows used to fit the comparison models also tempers causal attribution to tuning.

The displayed confidence/probability is a model output, not the model's accuracy and not a guarantee. The API code does not implement a separate probability calibration step. The code's local feature ranking multiplies built-in feature importance by the absolute transformed input value; it is a heuristic indicator, not a causal explanation or SHAP attribution. In particular, Product_variant_A is a one-hot representation of fleet category and should not be presented as a physical maintenance cause.

7. System process and graphs

What the system does: it uses FastAPI for its web API and React with Vite for the webpage [5], [6]. When the API starts, it loads the saved data-preparation pipeline and models. It can return one prediction, predict from a CSV file, compare models, and provide model and feature details. It also serves the built webpage. The files show a local demonstration app; they do not show that it is deployed as an online utility service.

The evaluation pipeline produces confusion-matrix, ROC and precision-recall plots, plus XGBoost feature-importance output. Figures 4 and 5 in this report are rebuilt from the checked metrics JSON so that plotted values match the summarized comparison. ROC-AUC should be presented alongside precision, recall, F1 and the confusion matrix rather than used alone.

8. Timing and system performance

Activity	Time	Evidence / limitation
XGBoost hyperparameter search	21.8 seconds	Recorded in models/model_metadata.json; 216 candidates × 5 folds.
Total model training pipeline	24.25 seconds	Recorded in metadata; includes the training call as measured by the training script.
Implementation / development time	Not recorded	Cannot be reconstructed reliably from project files.
Final test evaluation time	Not recorded	The 750-row test sample and scores are recorded; elapsed evaluation time is not.
Prediction / API response time	Not recorded	No benchmark log is present; avoid claiming a latency figure.

What was timed: the project saved the training times but did not save the computer model, memory, software versions, or a detailed timing log. The code sets settings for Apple ARM computers, but that does not identify the exact processor used for the recorded times. For a fair speed comparison, record the computer and software versions, time each training and testing step separately, and repeat prediction tests for single and batch inputs.

9. Limitations

- The dataset is semi-synthetic and research/education oriented; test performance does not establish field performance or safety suitability [1].
- The project split is stratified by product variant and target, but has no timestamp or asset identifier according to the local audit; temporal or asset-level generalization could not be assessed.
- Missing values are structured by product variant; imputation may not capture the operational meaning of unavailable sensors.
- Fine-tuned XGBoost and other models use different numbers of fit rows, weakening direct comparison.
- The project does not show how results vary across new data splits, how the model performs on a separate real-world dataset, which alert threshold best balances the costs of mistakes, whether probabilities match real outcomes, or how long development took.
- Feature ranking is not causal evidence. Dashboard wording should say “model indicators” or “important inputs,” with concise explanation and source where applicable.

10. Conclusion

CircuitGuard AI is a working demonstration that prepares data, trains models, compares results, and serves predictions through an API and webpage. On this test set, Random Forest has the best accuracy and F1. Fine-tuned XGBoost has the best ROC-AUC and recall. So XGBoost is not the best on every measure. It is a reasonable candidate if finding more maintenance cases and ranking them well are the main goals, but the data and training-size limits still matter.

Before these results guide maintenance work, the model needs to be checked on real breaker records, every model should be trained on the same rows, and the team should set a clear rule for when a prediction triggers inspection. The project should also record computer and run times. For now, the evidence supports a school or research prototype, not automatic maintenance decisions.

References

[1] S. Ghosh and G. Mittal, “Federated learning for critical electrical infrastructure-handling data heterogeneity for predictive maintenance of substation equipment,” Frontiers in Artificial Intelligence, vol. 8, art. 1697175, 2026, doi: 10.3389/frai.2025.1697175. Dataset and source description: https://doi.org/10.3389/frai.2025.1697175 and https://github.com/sghosh27/Federated-Learning-for-Critical-Electrical-Infrastructure-Maintenance

[2] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785-794, doi: 10.1145/2939672.2939785.

[3] XGBoost Developers, “XGBoost Parameters: Tree Booster,” official documentation. https://xgboost.readthedocs.io/en/stable/parameter.html (accessed Nov. 4, 2026).

[4] Scikit-learn Developers, “Metrics and scoring: quantifying the quality of predictions,” official documentation. https://scikit-learn.org/stable/modules/model_evaluation.html (accessed Nov. 4, 2026).

[5] FastAPI, “FastAPI documentation.” https://fastapi.tiangolo.com/ (accessed Nov. 4, 2026).

[6] React, “React documentation.” https://react.dev/ (accessed Nov. 4, 2026).

Project artifacts reviewed: training/preprocessing/evaluation scripts, inference and API code, saved model metadata, dataset source notes, data-quality report, and test metric JSON. Report date: November 4, 2026.